

EL 507 Embedded Control Systems II

Lab 12 Technical Report

Confederation College
Fateh Ali Shahrukh Khan
2025-12-14

1 Abstract

This lab report explains the design and implementation of a motor control system using PID controller and an H-Bridge circuit. The system regulates motor speed and direction by processing feedback from an incremental encoder based on two Hall effect sensors which provide quadrature signals (PhA and PhB) for speed and direction feedback to the MCU.

2 Introduction

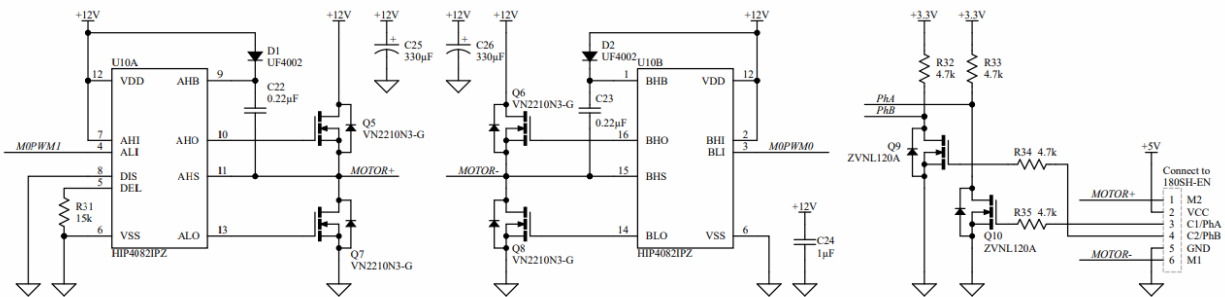


Figure 1: The complete motor controlling circuit

The circuit shown in Figure 1 uses an H-Bridge, gate driver, Quadrature Encoder and a motor connection to regulate the speed and direction of a motor.

a. H-Bridge circuit –

In order to control the direction in which the motor rotates, we can use two configurations, either unidirectional or bi-directional. As shown in Figure 2, we use a bi-directional configuration. In this configuration, four MOSFETs (Q5, Q6, Q7 and Q8) are connected in the shape of the letter ‘H’, hence the name.

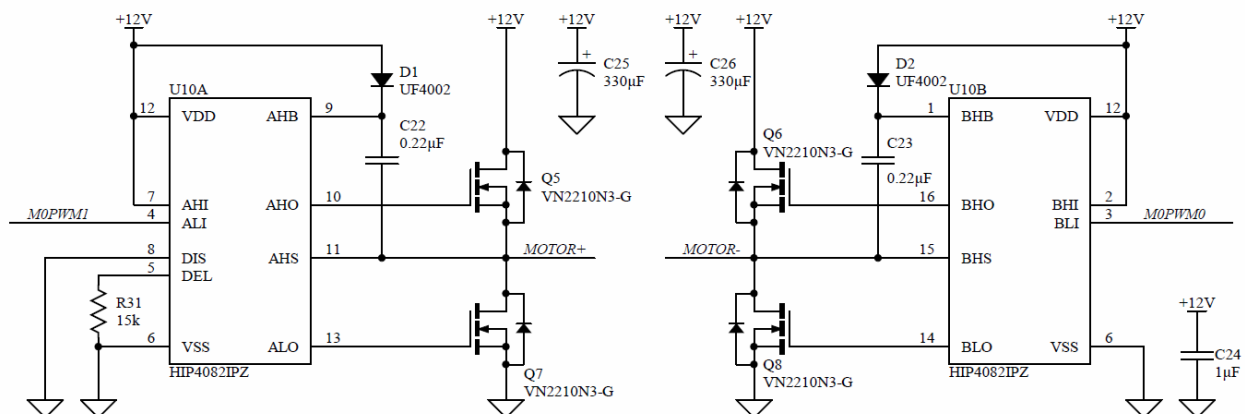


Figure 2: H-Bridge circuit

To rotate the motor in a chosen direction, either Q5 and Q8 are switched ON while Q6 and Q7 remain OFF, or Q6 and Q7 are switched ON while Q5 and Q8 remain OFF. In both cases, Q5 and Q6 are turned OFF and Q7 and Q8 are activated to safely dissipate the motor's magnetic field when the motor is powered down.

Another key component in the H-Bridge circuit is HIP4082IPZ IC. HIP4082IPZ is a 16-pin H-Bridge N-Channel MOSFET driver IC. It is used to independently drive the four N-Channel MOSFETs in either a half-bridge or full-bridge configuration.

b. Quadrature Encoder Interface (QEI) –



Quadrature Encoder is used to convert linear or rotary displacement into its equivalent digital signal that is used by the MCU to process.

c. Motor connection –

As shown in Figure 3, the output of the H-Bridge circuit (MOTOR+ & MOTOR-) is connected to pins 1 and 6 of 180SH-EN motor. The 180SH-EN motor has a no-load speed of 16000RPM, but for practical purposes, the maximum speed is 12000 RPM. The motor IC is powered by a +5V power supply that is connected to pins 2 and 5. Pins 4 and 3 are connected to

the gate of MOSFETs Q9 and Q10 whose operation is controlled by the PhA and PhB signals.

These signals are used to control the functioning of the motor.

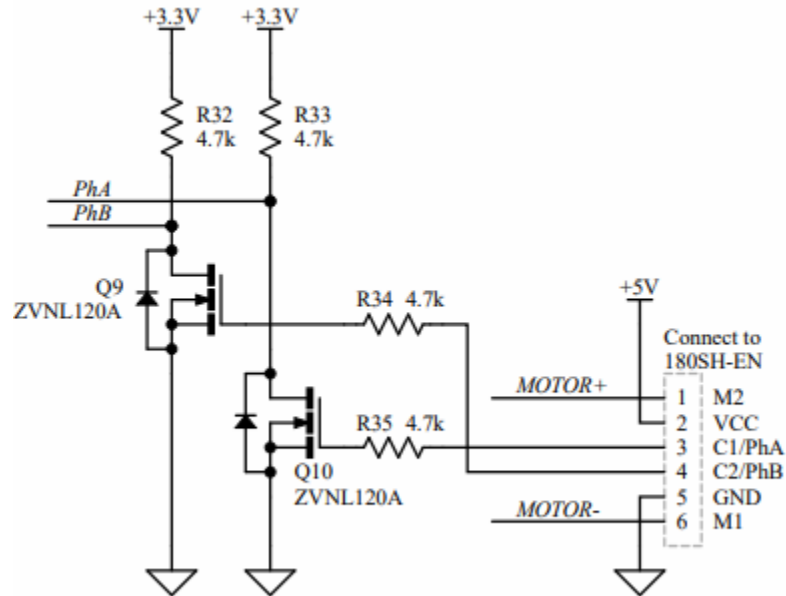


Figure 3: Motor connection

3 Theory of Operation

The MOTOR_PID() function is used to control the speed of the motor by calculating the appropriate duty cycle. To calculate the duty cycle, first the current speed of the motor is retrieved from the QEI, which is also known as the Process Variable (PV). Then, the error is determined by calculating the difference between the desired speed, or Set Point (SP), and the measured speed, or the Process Variable (PV).

$$\text{Error} = \text{Set Point (SP)} - \text{Process Variable (PV)}$$

Once the error is calculated, then the PID controller is used to fine tune the system to reduce the error to zero. To do that, the calculated error is used to determine the PID values, i.e,

- i. Proportional (P)
- ii. Integral (I)
- iii. Derivative (D)

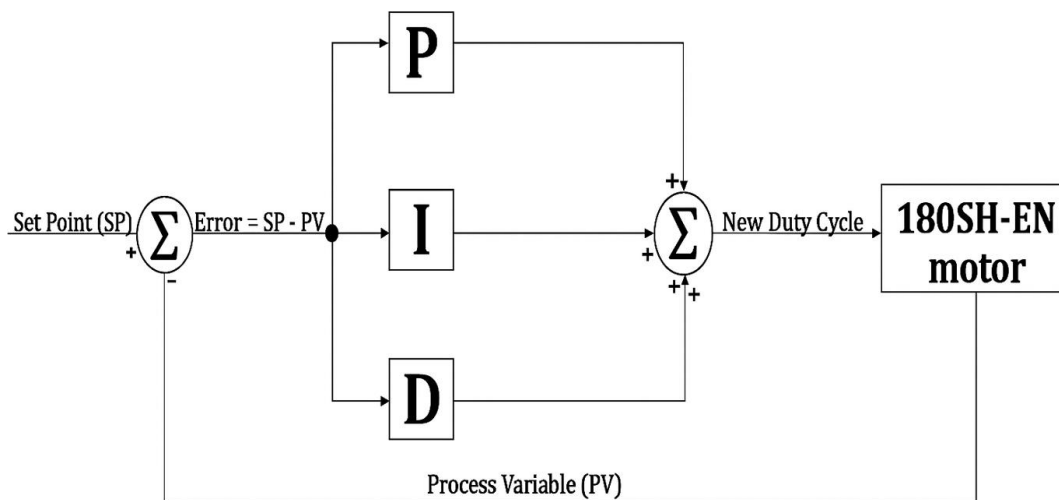


Figure 4: PID Control

Once the PID values are determined, they are summed to calculate the adjustment value. Using the adjustment value, the new duty cycle is calculated. Using the new duty cycle the speed of the motor is adjusted.

4 UML Diagram

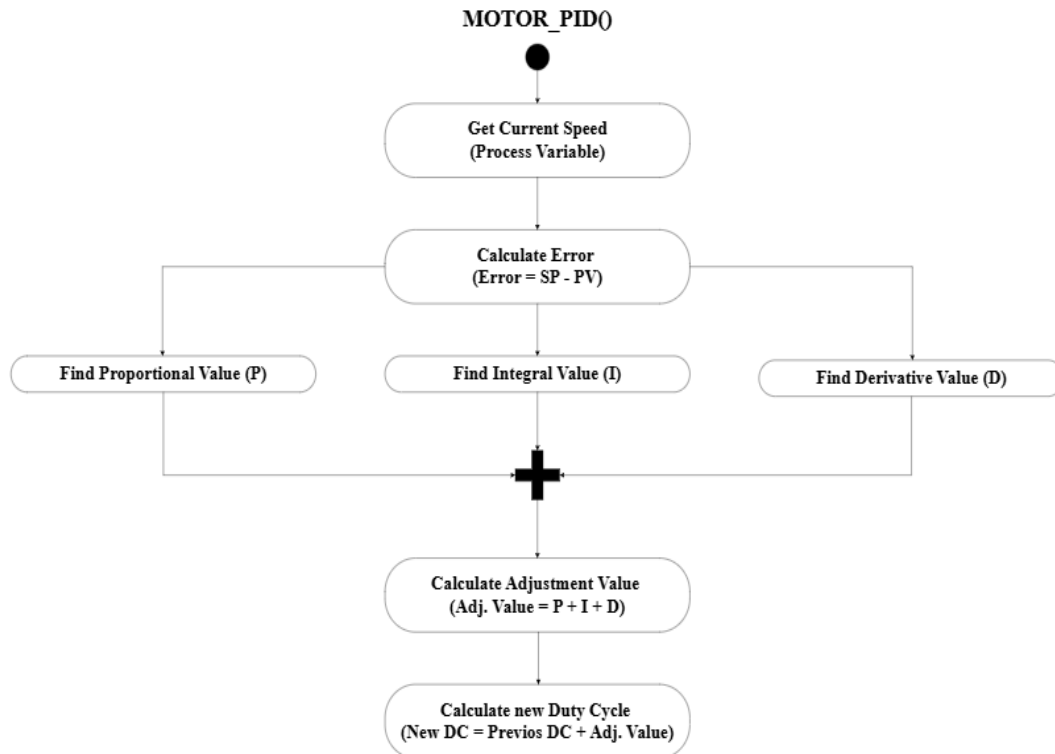


Figure 5: PID Control UML Diagram

5 Code Listing

```
void MOTOR_PID( MOTOR_CONTROL_PARAMS *pMCP )
{
    if( pMCP->fdt > 0.0f )
    {
        // Get the current speed
        pMCP->fPV = QEI_GetSpeed();

        // Determine error
        float fError = pMCP->fSP - pMCP->fPV;

        // Proportional
        float fPout = pMCP->fKP * fError;

        // Integral
        pMCP->fIntegral += fError * pMCP->fdt;
        float fIout = pMCP->fKI * pMCP->fIntegral;

        // As an option, if the error is within an acceptable limit, the
        // accumulated error (pMCP->fIntegral) can be zeroed.

        // Derivative
        float fDerivative = ( fError - pMCP->fPrevError ) / pMCP->fdt;
        float fDout = pMCP->fKD * fDerivative;

        // Update previous error
        pMCP->fPrevError = fError;

        // Set motor speed
        MOTOR_SetDutyCycle( fPout + fIout + fDout, pMCP->bDir );
    }
    return;
}
```

Code Listing